

1. Main Research Story

The current project direction is:

- > Use hMETIS as a structure-aware refinement tool for the Maximum Coverage Problem
- > on hierarchical community-structured hypergraphs.

The claim is not that hMETIS universally beats greedy. The more accurate claim is:

- > On graphs with community and subcommunity overlap structure, hMETIS-refinement
- > can improve greedy by finding structurally diverse replacement candidates.
- > Tabu search still gives the best final quality, but hMETIS-refinement captures
- > part of tabu's improvement with lower runtime.

2. Natural Hierarchical Graph

The graph is generated in 'experiment_natural_hierarchy.py'.

The vertices are grouped into hidden communities and subcommunities:

community
subcommunity
vertices

For example, in the large setting:

nvtxs = 4800
n_communities = 12
subcommunities_per_community = 5

This means:

12 communities
5 subcommunities in each community
60 total subcommunities
about 400 vertices per community
about 80 vertices per subcommunity

Communities and subcommunities are hidden generation labels. They are not selectable edges and they do not directly cover anything.

3. What Communities Are Used For

Communities and subcommunities are only sampling pools.

They tell the generator where an edge should sample vertices from.

community/subcommunity = hidden source pool

generated hedge/edge = actual selectable coverage set

Example:

Subcommunity A = {1, 2, 3, ..., 80}

Generated local edge = {3, 7, 11, 14, ..., 72}

The algorithm can select the generated local edge. It cannot select the subcommunity itself.

4. Edge Types

The generator creates four edge types.

Global Generalist Edges

Global edges sample a small number of vertices from many communities.

They represent broad actions such as:

- large campaign
- wide-area service
- general recommendation
- broad sensor/facility

Global does not necessarily mean largest. It means the edge is diverse across communities.

Sampling behavior:

- choose about 65% to 100% of all communities

- from each chosen community:

 - sample about 7% of popular vertices

 - sample about 1.2% of tail vertices

Community Edges

Community edges focus on one community, but sample across several subcommunities inside it.

Sampling behavior:

- choose 1 community

- choose 2 to all subcommunities inside that community

- from each chosen subcommunity:

 - sample about 18% to 35%

 - also sample about 3% of popular vertices from that community

Local Specialist Edges

Local edges focus deeply on one subcommunity.

Sampling behavior:

choose 1 community

choose 1 subcommunity

sample about 45% to 78% of that subcommunity

with 25% probability:

also sample about 6% of a neighboring subcommunity

Bridge/Noise Edges

Bridge edges connect two communities with small sampled pieces.

Sampling behavior:

choose 2 different communities

choose 1 subcommunity from each

sample a small amount from both

These edges add noise and cross-community overlap.

5. Edge Size Control

The generator controls edge size by sampling rules and a shared maximum cap.

The cap is:

$\text{max_edge_size} = \max(60, \min(220, \text{int}(0.055 * \text{nvtxs})))$

So approximately:

small: $\text{nvtxs} = 1200 \rightarrow$ max edge size about 66

medium: $\text{nvtxs} = 2400 \rightarrow$ max edge size about 132

large: $\text{nvtxs} = 4800 \rightarrow$ max edge size about 220

No edge type is guaranteed to be larger than another.

The edge type controls coverage pattern:

global = wide and shallow

community = medium-scale within one community

local = narrow and deep

bridge = small cross-community

All edge types are clipped if they exceed 'max_edge_size'.

6. Stochasticity

The graph is stochastic because every run randomly changes:

- which communities a global edge touches
- which vertices are sampled
- which community a community edge chooses
- which subcommunities a community edge touches
- how deep each local edge is
- whether a local edge touches a neighbor
- which two communities a bridge edge connects

So every instance has the same broad structure, but not the same exact edges.

This is more natural than the tailored bait graph because there is no exact known optimum and no hand-coded winning solution.

7. Why Greedy Can Struggle

Greedy repeatedly picks the edge with the largest current uncovered gain.

On this graph, greedy can over-select:

- broad global edges
- community edges that cover popular/common regions

These choices look good early, but they can overlap later. Greedy cannot undo earlier decisions.

The result is usually not a huge failure, because greedy is strong. But it can leave modest improvement opportunities.

8. Why hMETIS Helps

hMETIS partitions vertices, but MCP selects edges. To use hMETIS, the graph is written in reverse:

- hMETIS vertex = original MCP edge/set
- hMETIS hyperedge = original MCP vertex/element

If two MCP edges cover many of the same vertices, they appear together in many hMETIS hyperedges. That makes them structurally connected.

In the natural hierarchy graph:

- local edges from the same subcommunity overlap
- community edges inside the same community overlap
- global edges overlap on popular vertices
- bridge edges connect communities

hMETIS can detect this structure indirectly through overlap. It is not given

community labels.

9. hMETIS-Refinement Algorithm

hMETIS-refinement uses hMETIS as a candidate generator, not as the final solver.

It has two phases.

Phase 1: hMETIS Candidate-Pool Construction

```
start with empty selected set
while budget is not full:
    write current residual hypergraph to hMETIS
    partition remaining candidate edges
    take top candidates from each partition
    choose globally using current marginal coverage gain
    update selected edges and covered vertices
```

This is not simply "pick one edge from every partition." The partition only creates a diversified candidate pool. Marginal coverage still decides selection.

Phase 2: hMETIS-Guided Swap Refinement

After the initial solution is full:

```
repeat for several rounds:
    run hMETIS again
    collect replacement candidates from partitions
    for each candidate edge not currently selected:
        for each selected edge:
            compute exact coverage change if swapped
        accept the best positive-gain swap
    stop when no positive-gain swap exists
```

The most important rule:

accept a swap only if actual MCP coverage improves

So hMETIS proposes candidates, but the MCP objective makes the final decision.

10. Why Refinement Is More Reliable Than Direct hMETIS Selection

The original direct hMETIS method tries to use partitions too directly.

That can fail because:

- hMETIS minimizes cuts; MCP maximizes covered vertices
- partition-local choice may ignore global marginal gain
- early mistakes cannot be repaired
- one-per-partition constraints can reject good choices

hMETIS-refinement is safer:

- hMETIS only proposes candidates
- coverage gain accepts or rejects each swap
- bad hMETIS suggestions are ignored
- earlier selected edges can be replaced

11. Difference From Tabu

Tabu is a broader local search:

- starts from greedy
- tries many possible swaps
- can accept non-improving moves
- uses memory to avoid cycling
- keeps the best solution found

hMETIS-refinement is narrower:

- starts from hMETIS candidate-pool solution
- uses hMETIS partitions to propose replacement candidates
- accepts only improving swaps
- does not accept worse moves

Therefore tabu usually has better final quality, but hMETIS-refinement is faster and more directly tied to the hypergraph partitioning idea.

12. hMETIS Parameters

The main hMETIS-refinement configuration is:

```
nparts = 16
top_per_partition = 4
min_gain_ratio = 0.70
refine_top_per_partition = 10
refine_rounds = 5
timeout = 45 seconds
```

```
### nparts = 16
```

This means hMETIS partitions the candidate edge-space into 16 parts.

Why 16?

- small enough to keep runtime manageable
- large enough to expose multiple communities/subcommunities
- stable across small, medium, and large tests

Using too few partitions gives weak diversity. Using too many partitions

increases runtime and may make partitions too small/noisy.

For larger future tests, 'nparts = 32' is worth trying, but for the current focused comparison 'p16' is the clean default.

```
### top_per_partition = 4
```

During construction, each partition contributes up to 4 candidate edges.

Why 4?

1 candidate per partition is too restrictive
too many candidates makes hMETIS guidance less meaningful
4 keeps a small but flexible candidate pool

```
### min_gain_ratio = 0.70
```

A candidate must have at least 70% of the current global best marginal gain to be considered during construction.

Why?

prevents selecting very weak edges just for diversity
still allows hMETIS to choose slightly less greedy but more diverse edges

```
### refine_top_per_partition = 10
```

During refinement, each partition can contribute up to 10 replacement candidates.

Why more than construction?

refinement needs a broader pool to find useful swaps
bad candidates are safe because swaps must improve actual coverage

```
### refine_rounds = 5
```

The algorithm repeats hMETIS-guided swap search for up to 5 rounds.

Why 5?

enough to find multiple improvements
avoids very long runtimes
usually stops early if no improving swaps exist

```
### timeout = 45 seconds
```

Each hMETIS call has a timeout to avoid hanging on larger instances.

13. Current Comparison Result

On the natural hierarchy graph:

small:

greedy avg = 877.33

hMETIS-refine avg = 880.67

tabu avg = 883.33

medium:

greedy avg = 2009.67

hMETIS-refine avg = 2022.00

tabu avg = 2028.00

large:

greedy avg = 4465.33

hMETIS-refine avg = 4497.67

tabu avg = 4511.00

Runtime:

large:

greedy avg = 0.62s

hMETIS-refine avg = 44.60s

tabu avg = 165.77s

14. Best Final Claim

The most defensible claim is:

- > Tabu gives the best final solution quality. However, hMETIS-refinement gives a
- > faster structure-aware improvement over greedy. On larger hierarchical
- > community graphs, it captures a substantial portion of tabu's improvement
- > while using less runtime.